Static verification of RTK CRM

English version of ../VERIFICATION_REPORT.md. Date: 2026-09-22 Branch: dev Repository: github.com/Pavel1778/rtk-crm

The source code, the configuration, the documentation, the generation of exports, the frontend build and an approved Locust run were verified. A browser smoke test and Docker Compose were not run.

Update after the performance/UI block

Later commits on dev:

Commit	What was done
4cac43e	A Redis/in-process TTL report cache, invalidation after mutations, indexes and the removal of an N+1
853de15	A separate ReportsUser scenario for parallel reports
0698e76	JWT/Keycloak documentation, MOCK_MODE, production URLs and project links
354f67c	Vite manual chunks and migration of legacy UI colours to --atmr-* tokens
0df43eb	Fixed the ReportsUser selection in the Locust CLI (weight=1)

The actual local run on the SQLite backend:

- 50 users / 60 seconds: 1613 requests, 0 errors, aggregate p95 93 ms;
- 10 ReportsUser users / 30 seconds: 322 requests, 0 errors, p95 /api/reports 7 ms, aggregate p95 77 ms.
- 50 users / 3 minutes, mixed read/write: 4673 requests, 0 errors, p95 create 32 ms, move 43 ms, comments 45 ms, upload 33 ms, XLSX 48 ms, XLS 76 ms, PDF 210 ms, aggregate p95 110 ms;
- 10 users / 1 minute, exports only: 308 requests, 0 errors, p95 XLSX 37 ms, XLS 60 ms, PDF 180 ms;
- 10 administrators / 30 seconds, stage toggle: 308 requests, 0 errors, p95 disable 17 ms, enable 16 ms.

These values belong to the local SQLite environment and are not an SLA of production Render/Supabase.

Result

- □ OK — 35 items;
- ⚠ Warning — 8 items;
- □ Bug — 0 open items.

The fixable defects found were closed by commits:

Commit	What was fixed
`9611446`	RBAC for reference data and imports, production `SECRET_KEY`, CORS origins, MIME/extension checks, audit, export names, an export test, a system font for PDF, removal of a secret from Compose
`56d3fe0`	a root `SECURITY.md`, an up-to-date documentation architecture, responsive/theme tokens, a chart legend and sizes
`b69b813`	report filters by period, stage, university, product, direction and assignee; export filters
`4155019`	removal of hardcoded colours in the reports
`9643426`	removal of credential defaults from the deployment files and of secrets from the deployment documentation
`7324df1`	React Query caching for the reports page

Update after stages A and B (23.09)

Commits on `dev`:

Commit	What was done
stage A	PDF report: dynamic column widths, wrapping of long values, landscape with more than 5 columns, DejaVu Sans; CORS with an exact origin and `expose_headers`; a single range date picker with presets
stage A (follow-up)	a date range filter added to the Kanban board (`date_from`/`date_to` in `/api/interactions/board`), the end of the period included in full, an inverted range gives 400
stage B	an adaptive search magnifier for the university; percentages on the slices of the "Product share" chart; a single stage table component
`7ccbbc0`	stage reordering: a two-pass order update instead of a direct assignment, removing a 500 caused by `UNIQUE(scope, order)`
`017054c`	a single adaptive `StageTable` for the workflow constructor and the admin settings
`d32a1f0`	a single report column config for the interface and the PDF/XLSX/XLS/JSON exports

Checks after stages A and B:

- `python -m pytest` — 48 tests, all green (including the new sets `test_pdf_layout.py`, `test_cors.py`, `test_report_columns.py` and three regression tests for stage reordering);
- `npx tsc --noEmit` — 0 errors; `npm run build` — successful;
- browser check (Chromium, Playwright): the stage table at 375px shows cards, at 768px and 1440px a table, with no horizontal overflow; the reports page at 375px uses short column headings (as in the PDF), at 1440px the full ones (as in the XLSX);
- dragging stages goes out as a single `POST /api/stages/reorder` request and returns 200, the order is preserved in the database.

Update after stage C (23.09)

Commits on `dev`:

Commit	What was done
`3dde8ca`	structural problems of import validation and the XLSX report generator
`de4002d`	the `POST /api/catalogs/import/report` endpoint
`e0633d7`	fixed file upload from the interface (Content-Type for FormData)
`896535b`	a verification summary and a problem table in the import window
`d1bc614`	the buttons "Import valid only / Cancel / Download the report"
`a465903`	a mixed-import test with a skipped-row counter
`d7b1077`	the guide rendering service to PDF and the build of `frontend/public/docs/*.pdf`
`579bdd2`	the "Help" tab: a table of contents with anchors, search, PDF download

C.1. Verifying the correctness of the import

What was checked:

- `python -m pytest` — 60 tests, all green (`tests/test_import_report.py` was added: 8 unit validation tests and 4 endpoint contract tests — 401 without a token, an XLSX with counter headings, 400 on an unknown `catalog_type` and on a non-Excel file, importing only valid rows);
- `npx tsc --noEmit` — 0 errors; `npm run build` — successful;
- browser check (Chromium, Playwright) on `/directories`: a file of 4 rows (1 error, 2 warnings) opens the import window, the summary shows "Rows in the file: 4", "Errors: 1", "Warnings: 2", and the problem table contains the row, field, reason and level; the buttons "Import valid only (3)", "Cancel" and "Download the error report (XLSX)" are available. The import creates 2 records, the toast reports "Records imported: 2; rows skipped: 1", the window closes and there is no horizontal overflow.

An important fix: the global `Content-Type: application/json` header overrode the multipart boundary, and importing reference data through the interface answered `422 Field required: file`. The header is now removed for `FormData`, so the import scenario works end to end.

C.2. The "Help" tab: navigation, search and PDF

What was checked:

- `python -m pytest` — 65 tests, all green (`tests/test_guide_pdf.py` was added: 5 tests on the converter — every section of `USER_GUIDE.md` and `ADMIN_GUIDE.md` reaches the PDF, Cyrillic and inline markup are not lost, images are embedded as XObjects, a missing image does not break the render);
- `npx tsc --noEmit` — 0 errors; `npm run build` — successful;
- `python scripts/render_guide_pdfs.py` — builds `user-guide.pdf` (4 pages) and `admin-guide.pdf` (3 pages) into `frontend/public/docs/`;
- browser check (Chromium, Playwright) on `/help`: the table of contents contains 10 sections, the search filters the list by headings, a click on a contents item scrolls the document to the right heading, switching to the administrator guide rebuilds the table of contents, the "Download PDF" button is available, 5 guide images are rendered, and there is no horizontal overflow or console error.

An important fix: heading `id`s started with a digit (`#3-dashbord-i-navigaciya`), so `querySelector` failed with a `SyntaxError` and navigation through the table of contents did not work. The anchors received the `section-` prefix. Along the way a single Markdown image is no longer wrapped in `<figure>` inside `<p>` — the `validateDOMNesting` warning is gone.

1. Exports and reports

Check	Status	Evidence
XLSX is generated through `openpyxl`	✅ OK	`backend/services/excel_export.py:6-11,30-107`
XLS is generated through `xlwt` with a named colour	✅ OK	`backend/services/excel_export.py:13-17,110-157`
PDF is generated by ReportLab	✅ OK	`backend/services/excel_export.py:19-27,160-193`
Cyrillic in the PDF	✅ OK	`backend/services/excel_export.py:175-193`; `backend/Dockerfile:7-10`
PDF landscape and repeated table header	✅ OK	`backend/services/excel_export.py:166-173,235-248`
Mandatory export columns	✅ OK	`backend/services/excel_export.py:39-48,118-127,211-220`
`Content-Disposition` and CORS expose headers	✅ OK	`backend/api/reports.py:351-359,388-397,425-433`; `backend/main.py:63-70`
ASCII names `rtk-report-YYYYMMDD.ext`	✅ OK	`backend/api/reports.py:33-34`
Bearer protection of `/api/reports*`	✅ OK	`backend/api/reports.py:74-85,326-336,363-373,400-410`; a direct request without a token gives 401 as expected
In-memory export	✅ OK	`scripts/export_smoke.py`; produced XLSX 5 KB, XLS 5 KB, PDF 47 KB; the XLSX reads back with `openpyxl`, the PDF has `%PDF` and DejaVu

2. Chart data and filtering

Check	Status	Evidence
`by_stage` contains all active stages, including zero ones	✅ OK	`backend/api/reports.py:140-166,192-200`
`by_product` does not lose interactions without a product	✅ OK	`backend/api/reports.py:168-190`; the category `Без продукта`
The dynamics fill missing dates with zeros	✅ OK	`backend/api/reports.py:202-243`
Filters by period/stage/university/product/direction/assignee	✅ OK	`backend/api/reports.py:37-71,74-100`; `frontend/src/api/endpoints.ts:158-185`
The UI period filter and passing the filter to the export	✅ OK	`frontend/src/pages/ReportPage.tsx:32-62,117-152`
A vertical stage chart and label truncation	✅ OK	`frontend/src/pages/ReportPage.tsx:181-224`
The full stage name in the tooltip	✅ OK	`frontend/src/pages/ReportPage.tsx:210-216`
A product pie chart and a responsive legend	✅ OK	`frontend/src/pages/ReportPage.tsx:239-267`
Dynamics without dots and with `minTickGap=24`	✅ OK	`frontend/src/pages/ReportPage.tsx:278-300`
Loading/empty/error states	✅ OK	`frontend/src/pages/ReportPage.tsx:63-86,170-176,235-237,273-275`

3. Functional specification 1–13

Item	Status	Evidence / note
1. Catalogues and XLS/XLSX import	✅ OK	`backend/api/catalogs.py:16-89`; mapping in `backend/services/excel_import.py`
2. Reports by period with filters	✅ OK	`backend/api/reports.py:74-100`; the period UI in `frontend/src/pages/ReportPage.tsx:117-152`
3. Filters university/direction/product/assignee	✅ OK	The API supports all four filters: `backend/api/reports.py:47-56`
4. Recharts and export	✅ OK	`frontend/src/pages/ReportPage.tsx:181-300`; export routes
5. Stage transitions and comments	✅ OK	`backend/api/interactions.py:139-176,328-345`; frontend drawer/comments
6. PNG/JPEG/PDF/ZIP/GZIP/RAR/DOC/DOCX/XLS/XLSX	✅ OK	`backend/api/files.py:20-50,74-98`
7. XLS/XLSX/PDF columns	✅ OK	`backend/services/excel_export.py:39-48,118-127,211-220`
8. JSON endpoint	✅ OK	The JSON route in `backend/api/reports.py` with `get_current_user`
9. Creating/updating the workflow	✅ OK	`backend/api/stages.py:76-135`; the mutation is available to admin
10. Keycloak or MOCK_MODE	⚠ Warning	The current working contour uses JWT+bcrypt: `backend/auth/security.py:15-64`; Keycloak is present only in the legacy/target Compose
11. RBAC user/manager/admin	✅ OK	`backend/auth/security.py:67-88`; reference data and catalogues are protected for manager/admin
12. Web UI	⚠ Warning	The code and the production build were verified; a browser smoke test was forbidden by the current task
13. React Query caching	✅ OK	`frontend/src/main.tsx:13-21`; the reports use `useQuery`: `frontend/src/pages/ReportPage.tsx:34-49`

4. Security and privacy

Check	Status	Evidence
JWT expiry of 1440 minutes	OK	`backend/core/config.py:22-24`
bcrypt	OK	`backend/auth/security.py:15,25-30`
Production secret validation	OK	`backend/core/config.py:39-48`
Explicit CORS without a wildcard with credentials	OK	`backend/main.py:59-71`
User context in the audit	OK	`backend/auth/security.py:42-64`; `backend/middleware/audit.py:33-65`
Collection-level audit and redaction of secret fields	OK	`backend/middleware/audit.py:77-121`
MIME + extension + a 50 MB upload limit	OK	`backend/api/files.py:20-50,74-94`
SQLAlchemy ORM	OK	The backend API/services were checked; raw SQL only in fixed health/DDL statements
`.env` is untracked, examples are allowed	OK	`.gitignore`, tracked files `backend/.env.example`, `frontend/.env.example`
Old secrets in the Git history	⚠ `Warning`	The current files are clean. A check of all 952 blob objects found no real third-party credentials (Supabase/Render/GigaChat): only placeholders in the history. Rotating third-party keys is not required. Refined in `docs/CLEANUP_REPORT.md`: if the contour was brought up from `docker-compose.yml` without an override, only the local `SECRET_KEY` must be replaced

5. Workflow, RBAC and cookie consent

Check	Status	Evidence
All stages in the constructor	✅ OK	`frontend/src/pages/SettingsPage.tsx:285` loads `listStages(true)`, `StageTable` shows the stages of the selected funnel
Only active stages on the Kanban board	✅ OK	`frontend/src/pages/BoardPage.tsx` and `useStages('active')`
Optimistic update/ rollback	✅ OK	`frontend/src/hooks/useStages.ts:27-59`
409 with active interactions	✅ OK	`backend/api/stages.py:120-130`
Impact and transfer on deletion	✅ OK	`backend/api/stages.py:41-59,180-205`; `frontend/src/components/workflow/DeleteStageModal.tsx`
Disabled opacity 0.55	✅ OK	`frontend/src/index.css:401-404`
Cookie schema and necessary locked	✅ OK	`frontend/src/lib/cookieConsent.ts:1-94`; banner/settings components
Privacy routes and footer links	✅ OK	`frontend/src/pages/CookiePolicyPage.tsx`, `PrivacyPolicyPage.tsx`, `frontend/src/components/AppFooter.tsx:21-24`
The cookie banner on mobile	✅ OK	Manual check in DevTools (iPhone SE / Galaxy S20 / iPhone 14 Pro Max) — the banner takes 25–35% of the screen, the buttons stack, the modal stays within the screen

6. Non-functional requirements and documentation

Check	Status	Evidence / note
The production response target ≤1 s	⚠ Warning	The local run reached an aggregate p95 of 93 ms, but a production SLA has not been confirmed separately
50 concurrent users	✅ OK	A real local run: 1613 requests, 0 errors
10 concurrent report requests	✅ OK	A real `ReportsUser`: 322 requests, 0 errors, `/api/reports` p95 7 ms
The interface's write profile	✅ OK	4673 requests in 3 minutes: create/move/comment/upload/export with no errors
Toggling a workflow stage	✅ OK	308 requests: disable/enable through PATCH with no errors
SPA and viewport	✅ OK	`frontend/index.html:5`; React Router and the Vite build
The 400/401/403/404/409/422/500 codes	✅ OK	Checked statically across the FastAPI routes/handlers; the 422 handler: `backend/main.py:74-89`
USER_GUIDE and ADMIN_GUIDE	✅ OK	The files are present
SECURITY.md with 152-FZ and the Yandex Cloud migration	✅ OK	`docs/SECURITY.md:1-34`
DEPLOYMENT.md Render/Vercel/Supabase/Yandex Cloud	✅ OK	`docs/DEPLOYMENT.md`; credentials replaced with placeholders
ARCHITECTURE and C4 Mermaid	✅ OK	`docs/architecture/ARCHITECTURE.md`, `c4-context.md`, `c4-components.md`
Archi: functional and component (specification 6.7)	✅ OK	`functional.archimate` (the 5 path steps + services), `rtk-crm.archimate` (containers); ER — `er.archimate`. PDF: `functional.pdf`, `er-model.pdf`
An 11-slide pitch	✅ OK	`docs/pitch.md`
QA with 20 questions/answers	✅ OK	`docs/qa-jury.md`
Real screenshots in `docs/images/`	✅ OK	10 PNG files in `docs/images/`, inserted into the guides and the PDF through `scripts/render_guide_pdfs.py`

7. Atomaro UI and code quality

Check	Status	Evidence / note
`--atmr-*` tokens and theme classes	OK	`frontend/src/index.css:1-24`; the light class in `MainLayout.tsx`
Radii/spacing/responsive grids	OK	`frontend/src/index.css`; the report layout
Mobile table/export/chart handling	OK	responsive classes in `frontend/src/index.css`; a responsive legend in `ReportPage.tsx`
Footer safe area and max-content	OK	`frontend/src/components/AppFooter.tsx:8-15`
Hardcoded colours	OK	The working UI usages were moved to `--atmr-*`; HEX remains only in token definitions and native colour inputs
TypeScript build	OK	`source ~/.nvm/nvm.sh && npm run build`; the build completed, Vite only reported a bundle size >500 KB
Python compileall	OK	`python3 -m compileall -q backend/... scripts tests/load`
`git diff --check`	OK	Run after the final edits
print/TODO/console.log/`: any` scan	OK	A static `rg` scan found no matches in the checked directories

Update after the audit against the specification (23.09)

Commits on `dev`:

Com-mit	What was done
`f58b7a6`	access to interactions by id: a KAM works only with their own cards
`a8bf9c1`	user management in the interface (the "Users" tab) and the 152-FZ audit log (`GET /api/audit`, the "Journal" section)
`ef1069c`	file access inherits the interaction's access; the check is shared in `backend/api/access.py`

What was checked against the specification's business rules

- **"University + product" as a single interaction** — `_ensure_unique` checks both creation and a product change; a duplicate is rejected with `409`.
- **No more than two parallel interactions per university** — `_ensure_parallel_limit` is applied on creation and when a card is returned to the active ones.
- **Funnel scope** — a B2C card does not go into a B2B column and vice versa.
- **RBAC** — a KAM (`user`) is restricted to their own interactions for reading, updating, moving, deleting, comments and files; the executive and the administrator see everything. Admin navigation and pages are hidden from the other roles.
- **Assignee** — a KAM becomes the assignee on creation themselves; an executive/administrator picks any KAM in the creation window and can hand an existing card over.

Checks

- `python -m pytest` — 90 tests, all green (`test_audit_log.py` — 8, `test_interaction_access.py` — 14, including 5 file access scenarios);
- `npx tsc --noEmit` — 0 errors; `npm run build` — successful;
- `python scripts/render_guide_pdfs.py` — `admin-guide.pdf` rebuilt with the sections about users and the journal.

Update after the final audit (24.09)

Commits on `dev`:

Commit	What was done
`86d57b8`	CI: linter, types and tests in GitHub Actions, a shared code style
`3527936`	the audit log: filters by employee and period, a CSV export

What was added/closed

- **JSON import** — top-level validation against JSON Schema (Draft 7): an array of objects or `{"data": [...]}` is accepted; errors are returned with a path to the field; empty record objects and a missing mandatory `Название` are rejected. Covered by `tests/test_excel_import.py` (7 tests).
- **The seed script** — `scripts/seed.py` (the `--reference` flag) as a wrapper over `backend/seed.py`; the shared import bootstrap was moved into `scripts/_bootstrap.py`.
- **CORS** — the preview stand host is no longer hardcoded: the pattern comes through `CORS_ORIGIN_REGEX` and is disabled by default (see `docs/DEPLOYMENT.md`).
- **SAST/SCA** — `docs/security/SAST-SCA.md`: backend dependency pins, `npm audit` = 0 vulnerabilities, the residual `ecdsa 0.19.2` risk (PYSEC-2026-1325) accepted and justified (HS256 is used).
- **Architecture** — `docs/architecture/functional.md` and `er-model.md` added; `ARCHITECTURE.md` extended with a 300+ scalability section (KeyDB).

Checks

- `python -m pytest` — 98 tests, all green;
- `ruff check backend/ tests/ scripts/ conftest.py` — All checks passed;
- `mypy backend/` — no issues found in 40 source files;
- `bandit -c pyproject.toml -r backend/` — 0 issues;
- `npx tsc --noEmit` — 0 errors; `npm run build` — successful; `npm audit` — 0;
- `python scripts/seed.py` — reference data and demo data are created idempotently.

Update after the structure check (24.09)

A check of the project tree for divergences revealed two defects; both were closed.

1. The documentation mirror diverged from the sources

The "Help" tab serves Markdown from `frontend/public/docs/`, but that copy was maintained by hand and had fallen behind `docs/`:

- `ADMIN_GUIDE.md`, `SECURITY.md`, `ARCHITECTURE.md` contained earlier revisions (no B2C funnel, no report cache, no rate limit, no scheduler);
- `architecture/functional.md`, `er-model.md`, `security/SAST-SCA.md` and `STACK.md` were missing altogether, although the source documents link to them — internal links in Help led nowhere;

- the image `10-help-docs.png` differed from the original.

The mirror is now assembled deterministically: `scripts/sync_public_docs.py` copies the document set and rewrites relative links into absolute ones (`/docs/...`), because the page is rendered at the `/help` route. A repeated run does not change the tree. Drift is watched by `tests/test_docs_mirror.py` (6 tests): the synchronisation is reproducible, all links resolve, and there are no relative links or broken images.

2. The backend in the container could not find the report column config

`backend/services/report_columns.py` read the config as `Path(__file__).parents[2] / "config" / "report_columns.json"`. In the image the code lives in `/app` (the build context is `backend/`), so the path pointed at `/config/report_columns.json`, and the `config/` directory never reached the image — the XLSX/XLS/PDF exports and `/api/reports/columns` would have failed in Docker.

Fixed: the path is chosen from candidates (the repository root and `/config`) with support for an explicit `REPORT_COLUMNS_CONFIG`; a read-only mount `./config:/config` was added to both compose files. Covered by tests in `tests/test_report_columns.py`.

Checks

- `python -m pytest` — 110 tests, all green (was 98);
- `ruff check backend/ tests/ scripts/ conftest.py` — All checks passed;
- `mypy backend/` — no issues found in 40 source files;
- `npx tsc --noEmit` — 0 errors; `npm run build` — successful;
- `python scripts/sync_public_docs.py` — idempotent, the mirror matches `docs/`;
- `bash backend/start.sh` — comes up and answers `{"status":"ok"}` on `/api/health`.

Update: the run documentation (24.09, second pass)

Checking the commands from the README and `docs/DEPLOYMENT.md` for executability revealed that the documented local backend run did not work.

3. `uvicorn backend.main:app` could not start

The application is imported as the package `app` (`/app` in the container, `backend/` in the repository), while `backend/main.py` imports `app.*`. So `uvicorn backend.main:app` from the root failed with `ModuleNotFoundError: app`. That is exactly why the tests and `scripts/_bootstrap.py` create a temporary alias `app -> backend`.

`scripts/run_dev.py` was added — it makes the package importable and brings up uvicorn with autoreload (verified: `/api/health` answers 200). The README and `docs/DEPLOYMENT.md` were moved to it.

4. `backend/start.sh` used the same broken path

The Render script imported `backend.db.session` / `backend.seed` and ran `uvicorn backend.main:app`. It now creates the alias `app -> backend` (as in `conftest.py`) and works through `app.*`; verified by running it.

5. Divergences in the documentation

- `docs/README.md`: the project tree described a shape that does not exist (`backend/api/models/`, `backend/api/api/`, no `auth/`, `middleware/`, `services/`, `config/`); the technology table named FastAPI 0.104 (actually 0.141) and an active Keycloak 24.0 (actually JWT); the database initialisation command was `docker exec rtk_backend python seed.py` — in the container the code is in `/app`, so the correct form is `python /app/seed.py` or `python scripts/seed.py` from the repository.

- `README.md` and `docs/DEPLOYMENT.md`: `.env` was suggested to be placed in `backend/`, but `Settings` reads `.env` from the current directory — when started from the root the file was ignored.
- `README.md`: a link to `docs/ARCHITECTURE.md`, which does not exist (`docs/README.md` and `docs/architecture/` stand in for it).
- `docs/DEPLOYMENT.md`: it claimed that `start.sh` is the image CMD; in fact the CMD is `uvicorn app.main:app`, and the schema and reference data are created in the application lifespan.
- `docs/architecture/ARCHITECTURE.md`: Keycloak 24.0 was named as the active Auth Server; replaced with JWT + bcrypt with a note about the target contour.

Regressions are watched by `tests/test_dev_entrypoint.py`.

Critical: the Render deployment crash (24.09)

Production failed at start-up with `FileNotFoundError: '/config/report_columns.json'` and `Exited with status 1`. The analysis:

- `backend/services/report_columns.py` reads the config **at module import time**;
- Docker/Render build the image with the `backend/` build context, so the root `config/` never reaches the image;
- in Compose the path was covered by mounting `./config:/config:ro`, but Render has no mounts — `/config/report_columns.json` is absent, the `app.main` import fails and the container exits with code 1.

The fix:

- the canonical config is duplicated into `backend/config/report_columns.json` (`scripts/sync_report_columns.py`); the copy reaches the image;
- `report_columns.config_candidates()` checks the paths in order: the bundled `backend/config/` → the root `config/` → `/config`;
- `REPORT_COLUMNS_CONFIG` still overrides the path.

Verified by simulating the image layout (`app/` = only the contents of `backend/`, without the root `config/`): the imports of `app.main` and `PDF_COLUMNS` pass. Regressions — `tests/test_report_columns.py` (the bundled copy, the order of candidates, the idempotency of the synchronisation).

Additionally fixed:

- `scripts/test_export.py` → `scripts/export_smoke.py`: the script was named `test_*` (pytest does not collect it, since it lives outside `tests/`) and failed with `ModuleNotFoundError: app` — it imported `backend.services.*` without the bootstrap alias. It now uses `ensure_app_importable()` and runs.

Remaining

1. Rotate the credentials that previously reached the Git history; removing them from the current state does not revoke the old values.
2. Verify production latency on Render/Supabase separately; the local target has been reached.
3. If required, replace JWT with Keycloak or add an explicitly documented `MOCK_MODE`, should that be a mandatory condition of the commission rather than an alternative option.

Production incident 24.09: a database schema without `scope` (fixed)

Symptom (Render log): `Ошибка загрузки демо-данных: column workflow_stages.scope does not exist`; the production database was empty; `GET /` → 404.

Cause: the production schema was created by an early version of the application through `create_all(checkfirst=True)`, which does not alter existing tables. The Alembic migration described the schema but was never applied: `alembic.ini` sits in the repository root while the Docker build context is `backend/`, so the CLI is unavailable in the image. The production database was missing `workflow_stages.scope` and `interactions.scope`, and uniqueness was defined globally by `code`/`order`.

The fix (commit `5cf3855`, PR #13):

Change	What it does
`backend/db/schema_sync.py`	an idempotent reconciliation of the schema with the models: adds `scope` (`DEFAULT 'b2b'`) and indexes, replaces the global UNIQUE constraints with composite `(scope, code)`/`(scope, order)`; SQLite — recreation with data copying, Postgres — `ALTER TABLE`; marks the Alembic head revision
`backend/main.py`	`ensure_schema()` in the lifespan; `GET /` returns links instead of 404
`backend/start.sh`	the same initialisation order without Docker
`README.md`, `docs/DEPLOYMENT.md`, `LoginPage.tsx`	the real KAM demo account `kam@rtk.ru`/`kam123` instead of the non-existent `user@rtk.ru`

Verification after the deployment (24.09, Render/Supabase):

- `GET /` → 200 `{"service":"RTK CRM",...}`;
- `GET /healthz` → 200; `GET /readyz` → 200 (`database: ok`);
- stages: 18 (14 B2B + 4 B2C);
- universities: 5; interactions: 7 — the demo data was loaded.

Local verification: 125 tests green, `ruff`/`mypy` clean, `tsc` without errors; E2E on the legacy schema — the reconciliation was applied, `compare_metadata` without divergences.

Update: one-command start-up and SourceCraft CI (26.09)

The goal was for `docker compose up -d` to work on a clean machine and for the checks to run on the SourceCraft mirror as well.

1. `docker compose` did not build without environment variables

`docker compose config` failed with an interpolation error: compose requires `MINIO_ROOT_PASSWORD` and `KEYCLOAK_ADMIN_PASSWORD` (the `${VAR:?}` syntax), and there was no `.env.example` template in the repository root — only `backend/.env.example` and `frontend/.env.example`, which do not contain those names.

A root `.env.example` was added. The variable names were taken from `docker-compose.yml` and `backend/core/config.py` rather than from a third-party description: in particular, storage is configured through `MINIO_ROOT_*`/`S3_*` and not `MINIO_ACCESS_KEY`/`MINIO_SECRET_KEY`, and the GigaChat key is called `GIGACHAT_CREDENTIALS` rather than `GIGACHAT_AUTH_KEY`.

Verified: `docker compose --env-file <test> config` → exit 0, 7 services (`redis`, `minio`, `minio-init`, `backend`, `frontend`, `keycloak`, `nginx`).

2. Variables did not reach the backend container

Compose passed only `DATABASE_URL`, `REDIS_URL`, `S3_*`, `KEYCLOAK_URL`, `PYTHONPATH` and `CORS_ORIGINS` to the backend. Everything else, including `SECRET_KEY`, `AUTH_MODE` and `GIGACHAT_*`, never reached the

container: the application ran on the defaults from `config.py`, and switching `AUTH_MODE=keycloak` through `.env` was impossible.

Passing `SECRET_KEY`, `AUTH_MODE`, `KEYCLOAK_REALM`, `KEYCLOAK_CLIENT_ID`, `KEYCLOAK_AUDIENCE`, `GIGACHAT_*`, `SEED_DEMO_DATA` and the notification parameters was added. The Keycloak administrator login was moved into `KEYCLOAK_ADMIN_USERNAME` (it used to be hardcoded as `admin`).

Verified by parsing `docker compose config`: the values reach the service backend.

3. Migrations and seed need no manual run

`backend/main.py` in the lifespan performs `create_tables()` → `ensure_schema()` → `seed_demo()`. That is why a command like `alembic upgrade head && python -m scripts.seed && uvicorn ...` does not work in the container: `alembic.ini` sits in the repository root and never reaches the image (the build context is `backend/`), and `scripts/` is not copied into the image either. This had already caused the production incident of 24.09 (see the section above). The README now states that the initialisation is automatic.

4. CI for SourceCraft

`.sourcecraft/ci.yaml` was added with the same checks as in `.github/workflows/ci.yml`: `ruff`, `mypy`, `pytest` for the backend and `tsc --noEmit` + `npm run build` for the frontend. The paths match the repository structure — the tests live in `tests/`, not in `backend/tests/`.

Checking the containers

Previously the containers could not be started: the Docker daemon was unavailable and only the CLI was present, so the check had been static. Docker is up now, and the stack has been tested live.

What was run: `redis`, `backend`, `frontend`, `nginx` — built from the repository and brought up in one user network. The `minio/minio` and `minio/mc` images are unavailable in this environment (pull access denied), so the S3 storage was tested in the `S3_ENDPOINT=""` mode: files are written to the local `uploads/`. The `keycloak` container was checked separately while debugging `redirect_uri`.

Check	Result
`GET /healthz`	`200`, `{"status":"ok"}`
`GET /readyz`	`200`, `{"status":"ok","checks":{"database":"ok","cache":"ok","storage":"local"}}`
`GET /`	`200`, returns links to `docs`/`healthz`/`readyz`
Application start-up	the schema was reconciled (`ensure_schema`), the demo data loaded, the scheduler started
`POST /api/auth/login` (admin, manager)	a token was issued
`GET /api/auth/me` with a token	`200`, the role and `is_admin` match the user
`GET /api/universities` without a token	`401`
`GET /` through the frontend (nginx)	`200`, the application's `index.html` is served
SPA fallback (`/interactions`)	`200`
`/api/auth/me` through the frontend proxy	`200` — the same-origin scheme works
`/api/health`, `/health` through the root nginx	`200`

A defect in the root nginx was found and fixed: the config `nginx/nginx.conf` served `location /` from `/usr/share/nginx/html`, but the nginx image contains no built static files — a request to port 80 returned the default "Welcome to nginx!" page, although the README names `http://localhost` as the entry point. `location /` now proxies to the `frontend` container, as in the production contour on the VM (`infra/yandex-cloud/nginx-http.conf.template`).

A `readyz` with status `unavailable` on the cache only reproduces when the containers are started in the default Docker network: service names do not resolve there and the backend cannot see `redis`. In a user network the check passes. The VM uses a dedicated network, so this does not affect the product.

Update: the ER model, the presentation layout and the Q&A (27.09)

The final pass before submission. Three areas were checked separately, each against a real artefact rather than only against the code.

1. The ER diagram became reproducible

Previously the data schema lived only as text in `er-model.md`. Now the Mermaid block is extracted by `scripts/extract_er_diagram.py` into `docs/architecture/er.mmd`, and `mmdc` renders it into `er-model.pdf` and `er-model.png`.

Additionally an Archi file `docs/architecture/er.archimate` was assembled (ArchiMate 3, 5.0.0): 32 elements in the "Data (ER)" folder — 11 entities, 14 relationships and 7 Mini-Of-Entity. The file opens in Archi for free, and a ready PDF sits next to it for those who do not need Archi.

Check	Result
`scripts/extract_er_diagram.py`	writes `er.mmd`, 107 lines, starting with `erDiagram`
Rendering `mmdc -i er.mmd -o er-model.pdf`	a PDF of ~170 KB, one page
`er.archimate`	well-formed XML, `id="model-rtk-crm-er"`, 32 elements
Links in `ARCHITECTURE.md` and `architecture/README.md`	added, the mirror is in sync

2. The presentation layout was verified by rendering

`scripts/check_presentation_layout.py` was added. It converts the `.pptx` into a PDF through LibreOffice and looks for defects: text beyond the slide boundary, overlapping text blocks, a font size below 14pt on the mandatory slides 7–11, and a divergence of those slides' grid from the original template. Heuristics based on `python-pptx` are not used for finding overlaps — they produced false positives on the template's group shapes.

The mandatory LCT block now occupies positions 7–11 and repeats slides 7–11 of the template one to one: the geometry comparison passes with 0 defects. Verified at 960×540 pt: 11 slides, 0 layout defects. The metadata is clean: `author` and `last_modified_by` are "Сабадаш Павел".

3. The jury questions were split into two blocks

`docs/qa-jury.md` was reworked: part I covers 20 product questions (the problem, the user, Kanban, the 14 stages, access, data, 152-FZ, export, load, scaling, mobile, the next step), part II — 20 technical ones (integration, workflow versioning, audit, RBAC, integrity under a concurrent update, XLSX import, KeyDB, the LLM, Docker, documentation). Every answer references a concrete file instead of retelling the specification.

Conclusion

All three artefacts are reproduced with one command, the documentation mirror does not diverge from the sources (`tests/test_docs_mirror.py` — 6 passed), and the presentation has no visual defects when rendered.

Update: the cookie banner on mobile (27.09)

A check at real screen sizes revealed five cookie-consent defects that are visible only on a phone.

What was broken

Defect	Manifestation
The banner took over the screen	On an iPhone SE (375×667) the banner was 70% of the height, with buttons stacked in three rows
`column` + `flex-wrap`	Buttons moved into a second column beyond the screen
The banner's z-index of 1100	It overlapped the settings modal's footer: "Save the choice" could not be pressed
The modal was taller than the screen	4 categories + text did not fit, and the buttons ended up below the bottom edge
A hardcoded padding	`padding-bottom: 168px` was less than the real height of the mobile banner

What was changed

- `.cookie-banner`: the `z-index` was lowered to 900 (the antd modal is 1000, so the footer is reachable again); a `max-height: 55vh` limit with internal scrolling; `env(safe-area-inset-bottom)` for an iPhone with a notch.
- The mobile `.cookie-banner__actions`: `flex-wrap: nowrap`, a column, full-width buttons — three rows of labels are no longer clipped.
- The `.cookie-settings-modal` modal: at `≤575px` limited by `max-height: calc(100vh - 24px)`, the body scrolls, the footer is `column-reverse` with full-width buttons.
- `CookieConsentBanner.tsx`: the banner height is measured by `ResizeObserver` and written into `--cookie-banner-h`; `body.has-cookie-banner` takes the padding from that variable instead of a hard-coded value.
- The `.policy-card` policies: 14px text, the cookie category table scrolls locally (`scroll={{ x: 640 }}`), the headings are kept larger.

Verification

The check was performed in DevTools (responsive mode) on an iPhone SE, a Galaxy S20 and an iPhone 14 Pro Max: the banner size, the button layout, the modal, `localStorage` and the policy pages were compared. A run on the old build reproduces the defects; on the new one it passes:

```
before: iphone-se: the banner takes 70% of the screen
before: iphone-se/galaxy-s20/iphone-14-pro-max: the modal's save button is covered by another element
after:  OK
```

Device	Banner height before	after
iPhone SE 375×667	70%	35%
Galaxy S20 360×800	—	29%
iPhone 14 Pro Max 430×932	—	25%
iPad 820×1180	—	10%
Desktop 1440×900	—	9%

Additionally: no page has horizontal scrolling, the `body` padding matches the banner height on all checked routes, the modal and its footer are entirely within the screen, and the button contrast is 13.9 (dark) and 16.8 (light). A full backend test run — 200 passed.

Screenshots: `docs/images/mobile/cookie-{before,after}-*`, `policy-after-*`. The documentation mirror is synchronised (`python scripts/sync_public_docs.py`).

Repository synchronisation (27.09)

The work happens in two mirrors: `github.com/Pavel1778/rtk-crm` — the main one, and `git.source-craft.dev/lct-hackaton-2026/case-16-edtech-crm-team-54` — the mirror from which the working VM in Yandex Cloud pulls the code for the demo stand.

The working prototype: **https://crm-rtk.pixel-minds.ru** (a domain, HTTPS, Let's Encrypt).

The state at the time of the report:

Site	Branch	Note
GitHub	`main`	the source, every commit lands here
SourceCraft	`main`	the mirror the VM pulls the demo-stand code from

The synchronisation order is GitHub → SourceCraft: a commit first goes to the main mirror, then the same history is transferred to SourceCraft and from there pulled onto the VM. The SourceCraft token is never written into the repository or into `.git/config`: `GIT_ASKPASS` or a credential helper is used so that the secret does not settle in the working copy.

Anonymisation of the case holder's data

The case holder provided three exports: `Вендоры.xlsx` (a catalogue of IT products and vendors), `Данные оплат.json` (B2C requests) and `Загрузка пользователей.xlsx` (an individual's questionnaire). The questionnaire contains personal-data fields: SNILS, passport series and number, issued by and when, department code, registration address and postal code.

Before being added to the repository the questionnaire was **deliberately anonymised**: every personal-data field was replaced with a generated placeholder (`000-000-000 00`, `0000 000000`, `ОТДЕЛОМ УФМС РОССИИ`, `01.01.2020`, `000-000`, `г. Москва, ул. Примерная, д. 1, кв. 1`, `000000`). The full name, phone and e-mail were left as test data. The rebuild is `scripts/anonymize_case_data.py`. The original is stored outside the repository. This complies with 152-FZ and with the project's security policy (see `docs/SECURITY.md`, `docs/examples/README.md`).

The anonymised samples live in `docs/examples/` together with a mirror in `frontend/public/docs/examples/`: `vendors.xlsx`, `payments_b2c.json`, `users_b2c.xlsx`. The Latin naming keeps the files intact on the Linux stand and in the URLs of the "Help" tab.

Verified: `git log --all --full-history -- "*Загрузка*"` — empty, no real personal data in the history.

What the submission includes

- `docs/qa-jury.md` — 40 questions (20 product and 20 technical) grounded in code files rather than in a retelling of the specification.
- `docs/architecture/functional.archimate`, `functional.pdf`, `functional.mmd` — the functional model (specification 6.7): the 5-step user path and the services, an ArchiMate 3 model and the rendered diagram.
- `docs/architecture/rtk-crm.archimate` — the component model (specification 6.7): containers, external systems and the target deployment.
- `docs/architecture/er.mmd`, `er-model.pdf`, `er.archimate` — the ER model: the Mermaid source, the rendered diagram and the ArchiMate 3 model (32 elements).
- `docs/rtk-crm-documentation.pdf` — a single PDF (67 pages) with all the sections, including the ER and the jury Q&A.
- `docs/presentation/RTK-CRM-LCT2026.pptx` — the presentation, 11 slides (the presentation and its PDF are handed over to the organising committee; the repository holds the text version `docs/pitch.md`).
- `docs/examples/` — format samples from the case holder for demonstrating the import: `vendors.xlsx`, `payments_b2c.json`, `users_b2c.xlsx` (anonymised). Synchronised into the mirror.
- `frontend/public/docs/` — the documentation mirror for the "Help" tab (`scripts/sync_public_docs.py`, links rewritten to absolute `/docs/...`).

Remaining before submission

- **Deploying and checking the stand `https://crm-rtk.pixel-minds.ru`** — Postgres, Redis/KeyDB, MinIO, Keycloak, backend, frontend, nginx.
- **MinIO on the stand.** In the isolated build environment the MinIO images could not be pulled, so the prototype was tested on the S3 fallback with a local `uploads/`. On the VM MinIO comes up (`pgsty/minio`) — it must be confirmed that the backend writes attachments there and not to the local disk.
- **A load report on the target contour.** The local Locust run produced 1613 requests in 60 seconds, 0 errors, aggregate p95 93 ms. The figures on the VM must be re-verified and `tests/load/report.html` updated.

Final check (27.09.2026)

The final pass before submission: the presentation was brought to the organisers' requirements, and the submission links, the documentation and the tests were verified.

The presentation — the LCT block at positions 7–11

The organisers require slides 7–11 to keep the original template design. The mandatory block was moved to positions 7–11 and repeats slides 7–11 of the template one to one: the grid, the fonts and the colours were not changed, only the placeholders were filled in. Slides 1–6 (the solution) are designed freely.

Check	Result
Positions 7–11 against the template (`check_presentation_layout.py`)	the grid matches, 0 defects
Overflow/overlaps/font size on 7–11	0 defects, minimum 14pt
The team slide	5 cards; with fewer members the extra ones are removed
The demo domain on the slide	`https://crm-rtk.pixel-minds.ru`
PPTX	`docs/presentation/RTK-CRM-LCT2026.pptx`, 11 slides
PDF	`docs/presentation/RTK-CRM-LCT2026.pdf`, 11 pages
Font	Montserrat (Regular/Bold) substituted at render time, no fallback
PPTX/PDF metadata	`author` — "Сабадаш Павел", no traces of a generator

The four submission links

#	Link	Result
1	`github.com/Pavel1778/rtk-crm`	public: README, LICENSE (MIT), a CI badge
2	`docs/presentation/RTK-CRM-LCT2026.pptx` + PDF	handed over with the submission (see the presentation section)
3	`crm-rtk.pixel-minds.ru`	opens; the demo logins verified
4	`docs/rtk-crm-documentation.pdf`	67 pages

Verified without a session: `/help` — 200, `/docs/rtk-crm-documentation.pdf` — 200, `/docs/USER_GUIDE.md` — 200, `/health` — 200. The three demo logins answer 200 on `POST /api/auth/login`: `admin@rtk.ru`, `manager@rtk.ru`, `kam@rtk.ru`.

The README is closed against the checklist

It was verified that the README contains a link to the prototype, the demo login table, the `docker compose up -d --build` command and Swagger UI. What was missing was added: links to the presentation (PPTX + PDF) and a "License" section.

The divergence about `/docs` on the domain is explained separately: the `/docs` path is taken by the frontend's static documentation (`frontend/public/docs/`), so the browser gets the SPA there. Swagger UI is available locally at `http://localhost:8000/docs`.

Tests and linters

Check	Result
`pytest -q`	207 passed
`ruff check backend/ tests/ scripts/ conftest.py`	clean
`mypy backend/`	clean (48 files)
`tsc --noEmit` (frontend)	clean
`vite build` (frontend)	built
`sync_public_docs.py`	the mirror does not diverge

Update: B2B/B2C demo data and seed top-up (27.09)

The case holder's demo contour must show a portfolio of several universities with substantive tasks rather than a single card per stage. Along the way a top-up defect was fixed: previously `_seed_demo_interactions` returned immediately when the interactions table was non-empty, so a database seeded before the B2C funnel appeared stayed without B2C cards forever.

What was changed:

- B2B cards are defined by names (university, product, stage) instead of indexes; the indexes failed with an incomplete reference set and depended on the creation order.
- The seed creates the missing cards and tasks instead of skipping the whole set. The card key is the pair (university, product): the stage is not part of the key, otherwise moving a card to another column would produce a duplicate on restart.
- The demo composition: 10 active B2B cards (2 per university — the limit of the rule "no more than 2 active interactions per university", occupying 10 of the 14 stages) and 5 B2C requests; 7 tasks (4 open, 3 completed) and a comment.

Verification (locally, SQLite):

```
pytest -q                       -> 208 passed
ruff check backend/ tests/ ...  -> clean
mypy backend/                   -> clean
python scripts/seed.py (twice)  -> b2b 10, b2c 5; no duplicates
```

Update: the layout of the KPI tiles and the step numbers (27.09)

Two fixes after checking the interface and rendering the presentation:

- Reports, the KPI tiles. The grid had hardcoded 4 columns while there are five indicators: the fifth tile ("Completed tasks") stayed alone in the second row and looked clipped. The grid was moved to `auto-fit`: at 1440 and 1024 px all five tiles fit in one row, at 768 in two, on mobile in three.
- The presentation, slide 10. The corporate Montserrat is wider than the template's Arial, and in a 0.24" wide frame the step numbers "02" and "03" wrapped onto two lines ("0" and "2"). The numeric markers were switched to `wrap="none"` and their frame widened to 0.6"; the numbers "01", "02", "03" are on one line again.

Verification:

```
pytest -q                              -> 208 passed
ruff / mypy                            -> clean
npm --prefix frontend run build        -> built, tsc without errors
scripts/check_presentation_layout.py   -> 0 layout defects
```

A regression was added to `check_presentation_layout.py`: a step number wrapped onto two lines near the right edge of a slide now counts as a layout defect.